Rust Concepts — Three Levels of Understanding

Every concept from this repo explained three ways: like you're 5, like you're 15, and like a professional who needs to ship code. Each section links to the source file where you can see it in action.

> *This is the theoretical companion to your code. For deeper notes with worked examples, see* `notes.md` *.*

How to use this guide

Read the "Like I'm 5" first — always. Even if you're advanced, the analogy is the mental model. Then read your level. Come back to the professional level once you've written real code.

Before We Start: Two Concepts That Explain Everything

These two ideas — the **stack vs heap** and **what `&` means** — unlock every other concept in Rust. Read them first. The rest of the guide will make much more sense.

Foundational: Stack & Heap

> *There is no source file for this — it's how your computer works underneath every program.*

Like I'm 5

Your computer's memory is like a school. There's a **desk** (the stack) where you put things you're using right now — it's fast to grab stuff, but there's limited space and it gets cleared when you leave the room. There's also a **storage room** (the heap) — much bigger, you can store big things there, but you have to walk there and back to get them, and you're responsible for putting things away when you're done.

Like I'm 15

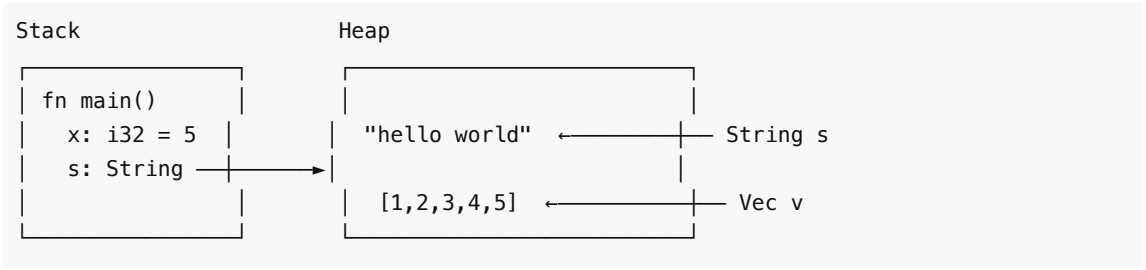
Every running program has two regions of memory:

The Stack — fast, automatic, limited

- Works like a stack of plates: last in, first out
- When you call a function, a new "frame" is pushed on top with all its local variables
- When the function returns, that frame is popped off — all those variables vanish instantly
- Values must have a **known, fixed size** at compile time
- Examples: `i32`, `f64`, `bool`, tuples, arrays — these all live on the stack

The Heap — slower, manual, flexible

- A big pool of memory your program can request at runtime
- Values can be any size — even sizes you don't know until the program runs
- You have to ask for memory, and then someone has to give it back when done
- In other languages (C), the programmer does this manually. In Rust, the **ownership system** does it automatically.
- Examples: `String`, `Vec<T>`, `HashMap` — these live on the heap



`x` (an `i32`) lives entirely on the stack — fast, small, known size. `s` (a `String`) has a tiny pointer on the stack, pointing to the actual text on the heap.

Professional

Stack allocation is a pointer bump — O(1) and cache-friendly. Heap allocation involves the allocator (typically `jemalloc` or system `malloc`) finding a free region, tracking metadata, and returning a pointer — notably slower and subject to fragmentation. Rust's ownership system makes heap allocation deterministic: when the owner goes out of scope, `Drop` is called and the memory is freed immediately — no garbage collector needed. `Box<T>`, `String`, `Vec<T>`, `Rc<T>`, and `Arc<T>` all represent heap-allocated values. Types that implement `Copy` are always stack-allocated (bitwise copy on assignment). This distinction is why Rust can be both memory-safe and have predictable performance.

Foundational: What `&` Means

> *This shows up constantly in `src/borrowing.rs`, `src/mut_borrowing.rs`, `src/arrays.rs`, `src/slices.rs`, and almost everywhere else.*

Like I'm 5

Imagine you wrote your phone number on a piece of paper. Instead of giving someone your phone, you give them the **paper with the address of your phone**. They can look up your phone number using that paper, but they don't own your phone. The `&` sign is how you give someone the "address paper" instead of the actual thing.

Like I'm 15

`&` creates a **reference** — a pointer to a value. Instead of handing the value itself (which would move or copy it), you hand a reference to where the value lives in memory.

```
let x = 5;
let r = &x;       // r is a reference TO x, not a copy of x
println!("{}", r); // Rust auto-follows the reference to print 5
println!("{}", *r); // * explicitly follows the reference ("dereference")
```

```
let name = String::from("Alice");
let r = &name;    // r points to the same String data as name
// name still exists. r is just another way to look at it.
```

`&` appears in three key places:

- **`&T`** — "I want to look at this value but not own it"

- **`&mut T`** — "I want to look at AND change this value but not own it"
- **`&str`** — a reference to a piece of text (string slice)

`*` is the opposite — it "follows" a reference to get the real value underneath.

Professional

`&T` creates an immutable borrow — a fat or thin pointer (depending on whether the referent is sized) that the borrow checker tracks. It does not change the owner; it creates an alias with a restricted lifetime. The compiler guarantees that the referent is valid for the entire lifetime of the reference. `&mut T` creates an exclusive borrow — no other reference (`&T` or `&mut T`) may exist simultaneously. This aliasing XOR mutation rule is the core invariant that makes Rust safe without a GC. Rust auto-derefs in method calls (`.` syntax) so explicit `*` is rarely needed except in arithmetic or assignment through a mutable reference.

---

1. Hello World & Entry Point

> *Source:* *src/main.rs*

Like I'm 5

When you turn on a toy robot, it starts doing things. `fn main()` is the ON button for your Rust program. `println!` is how you make the robot talk out loud.

Like I'm 15

Every Rust program needs a `main` function — that's where execution begins. `println!` is a macro (not a regular function) that formats text and writes it to the terminal. The `!` means it's a macro, not a function.

```
fn main() {
    println!("Hello, world!");
}
```

Macros look like function calls but are expanded by the compiler before your code even runs. `println!` is special because it checks your format string at compile time — you can't accidentally pass the wrong number of arguments.

Professional

`fn main()` is the binary entry point. `println!` is a format macro that expands at compile time — it validates format strings against arguments and calls `std::fmt::Display` under the hood. For library crates there is no `main` . For logging in production, prefer `eprintln!` to stderr or a structured logging crate like `tracing` .

---

2. Variables & Immutability

> *Source:* *src/variables.rs*

Like I'm 5

Imagine you write your name on a piece of paper and put it in a box labeled "name". In Rust, once the paper is in the box, nobody can change it unless you say so. The box is **locked by default**.

Like I'm 15

Variables in Rust are immutable by default. You use `let` to create a variable. The compiler figures out the type automatically (called *type inference*), so you often don't need to write it.

```
let x = 5;        // the compiler knows this is an i32
let y: i32 = 10;  // or you can say the type yourself
```

If you try to change `x` later, Rust refuses to compile. This isn't a crash — it's caught before your program ever runs.

Professional

Immutability by default enforces intentionality and eliminates a class of accidental mutation bugs at zero runtime cost. Type inference uses the Hindley-Milner algorithm; annotate types at API boundaries for documentation and to produce better compiler errors. `let _unused = value` suppresses unused-variable warnings without consuming the value.

---

3. Mutability

> *Source:* `src/mutability.rs` *— see* `sum_to()`

Like I'm 5

Sometimes you need a scoreboard that changes as the game goes on. You tell Rust "this number is allowed to change" by writing `mut`.

Like I'm 15

Add `mut` after `let` to make a variable mutable. The `sum_to` function in the source file does exactly this — it starts at 0 and adds numbers in a loop:

```
fn sum_to(n: u64) -> u64 {
    let mut sum = 0;      // sum needs to change each iteration
    for i in 1..=n {
        sum += i;
    }
    sum
}
```

Without `mut`, the line `sum += i` would be a compile error.

Professional

`let mut` is the only way to mutate a local binding. Rust distinguishes interior mutability (via `Cell` / `RefCell` / `Mutex`) from standard `mut` bindings. Prefer immutable bindings; reach for `mut` only when the value genuinely changes over time. The compiler can optimize immutable bindings more aggressively in hot loops.

---

4. Variable Shadowing

> *Source:* *src/shadowing.rs* — *see* *transform()*

Like I'm 5

Imagine you have a sticky note that says "5". You put a **new sticky note on top** that says "ten". Now when someone looks, they see "ten". The old note is still under there, but hidden.

Like I'm 15

Shadowing lets you reuse a variable name with a new `let`. You can even change the type this way — something `mut` alone can't do. The `transform` function in the source converts a string to a number using shadowing:

```
fn transform(s: &str) -> i32 {
    let s = s.trim();         // s is still &str, but trimmed
    let s: i32 = s.parse().unwrap(); // now s is i32 — same name, different type!
    s * 2
}
```

This is cleaner than inventing names like `s_trimmed` and `s_parsed`.

Professional

Shadowing creates a new binding in the current scope — it does not mutate the original. The old binding is dropped when the new one is created (or at end of scope if heap-allocated). Idiomatic for parsing pipelines. Unlike `mut`, shadowing can change the type. Avoid shadowing across large scopes where it hurts readability.

---

5. Primitive Types

> *Source:* *src/primitives.rs* — *see* *multiply()*

Like I'm 5

Numbers come in different sizes of cups. A tiny cup (`i8`) holds small numbers. A huge cup (`i128`) holds enormous numbers. Some cups only hold positive numbers (`u8`), some hold negative too (`i8`). If you pour too big a number into a tiny cup, it **overflows and makes a mess**.

Like I'm 15

Rust's built-in types:

Type	What it stores	Range
`i8`	small signed integer	-128 to 127
`i32`	standard integer (default)	~-2 billion to ~2 billion
`i64`	large signed integer	huge
`u8`	small unsigned (positive only)	0 to 255
`u64`	large unsigned	0 to ~18 quintillion

`f32`	floating point (decimal)	~7 decimal digits
`f64`	double precision float	~15 decimal digits
`bool`	true or false	`true`, `false`
`char`	a single character (Unicode)	any character

```
let a: u8 = 255;
let b = a.wrapping_add(1); // 0  — wraps around like a clock
let c = a.checked_add(1);  // None — safely says "it didn't fit"
let d = a.saturating_add(1); // 255 — stays at the max
```

Overflow **panics** in debug mode (so you catch bugs). In release mode it wraps silently.

Professional

Choose integer types by domain semantics, not performance (the compiler handles sizing). Use `usize` for indexing and lengths — it matches the pointer width of the target architecture. Use `checked_*`, `saturating_*`, or `wrapping_*` arithmetic in any domain where overflow is plausible. `f64` is the default float; use `f32` only when memory layout or SIMD demands it. `char` is a Unicode scalar value (4 bytes) — not a byte, not a UTF-16 code unit.

---

6. Tuples

> *Source:* `src/tuples.rs` *— see* `swap()`

Like I'm 5

A tuple is like a **lunchbox with separate compartments**: one for a sandwich, one for an apple, one for juice. Each compartment holds a different kind of thing, and the box always has the same number of compartments.

Like I'm 15

A tuple groups values of different types together. The `swap` function in the source returns two values at once using a tuple:

```
fn swap(a: i32, b: i32) -> (i32, i32) {
    (b, a)  // return both values packed into a tuple
}

let (x, y) = swap(1, 2); // unpack ("destructure") the tuple
println!("{}", x); // 2
println!("{}", y); // 1
// Or access by index:
let result = swap(1, 2);
println!("{}", result.0); // 2  (first slot)
println!("{}", result.1); // 1  (second slot)
```

Professional

Tuples are anonymous product types. They live on the stack when all fields are `Copy`. Use them for small, local groupings where naming a struct would be overkill — especially multi-value returns. For public API boundaries, prefer named structs for documentation and forward compatibility. Pattern matching on tuples is exhaustive and zero-cost.

---

7. Arrays & Slices

> *Sources:* *src/arrays.rs* — *`first_last()`* | *src/slices.rs* — *`first_word()`*

Like I'm 5

An **array** is like an egg carton — it has exactly 12 slots, always 12, never more or less. A **slice** is like pointing at some of the eggs: "I mean these three in the middle." You're not copying them — you're just describing where to look.

Like I'm 15

Arrays have a fixed size known at compile time: `[T; N]`. They live entirely on the stack. **Slices** (`&[T]`) are a "window" into an existing sequence — a reference to part of an array or vector.

```
// from src/arrays.rs
fn first_last(arr: &[i32]) -> (i32, i32) {
    (arr[0], arr[arr.len() - 1])
}

let nums = [10, 20, 30, 40, 50];
let (first, last) = first_last(&nums); // pass a slice of the whole array
```

Notice `&[i32]` in the parameter — this is a slice. It can accept an array, a Vec, or any contiguous sequence.

```
// from src/slices.rs
fn first_word(s: &str) -> &str {
    // &str is a slice of text — a "window" into a String
    s.split_whitespace().next().unwrap_or("")
}
```

Professional

Array type encodes size: `[i32; 3]` and `[i32; 4]` are distinct types. Slices are fat pointers (pointer + length). Prefer `&[T]` parameters over `&Vec<T>` or `&[T; N]` in function signatures — they're more general (accept both arrays and Vecs via deref coercion). Bounds checks happen at runtime; use `.get(i)` to return `Option<&T>` instead of panicking. `split_at` , `chunks` , and `windows` are powerful slice methods for parsing.

---

8. Functions

> *Source:* *src/functions.rs* — see *`max_of_three()`*

Like I'm 5

A function is like a **recipe**. You give it ingredients (inputs), it does something, and gives you back a dish (output). You can use the same recipe again and again without rewriting it.

Like I'm 15

Functions take typed parameters and return a typed value. The **last expression in a function without a semicolon** is automatically returned — no need to write `return`.

```
// from src/functions.rs
fn max_of_three(a: i32, b: i32, c: i32) -> i32 {
    if a >= b && a >= c {
        a
    } else if b >= c {
        b
    } else {
        c   // no semicolon = this is the return value
    }
}
```

You must always write the types on parameters and the return type. Rust never guesses function signatures.

Professional

Rust functions are always typed at the signature — no duck typing. The body is an expression block; the final expression (no semicolon) is the return value; a semicolon turns it into a statement returning `()`. Use `return` for early exits only. Functions are monomorphized when generic — there is no virtual dispatch cost unless you use `dyn Trait`. Associated functions (no `self`) are called with `::` and serve as constructors or static helpers.

---

9. If as Expression

> *Source:* *src/if_expr.rs* — *see* *abs_value()*

Like I'm 5

In Rust, an "if" can **hand you something**. Like asking "is it raining?" — if yes, take the umbrella; if no, take sunglasses. The answer *is* the thing you take.

Like I'm 15

`if` returns a value in Rust. Both branches must return the same type. This lets you assign the result of an `if` to a variable without a temporary `let mut`.

```
// from src/if_expr.rs
fn abs_value(n: i32) -> i32 {
    if n < 0 { -n } else { n }  // if is an expression — it produces a value
}
```

```rust
// You can also do this inline:
let label = if score >= 90 { "pass" } else { "fail" };
```

Professional

`if` is an expression — it unifies C's ternary operator with structured branching. All arms must have the same type; a lone `if` without `else` has type `()`. Prefer this form over temporary `let mut` followed by reassignment — it makes initialization unconditional, which lets the compiler prove the variable is always initialized.

---

10. Loops

> *Sources:* *src/loops.rs* — *`factorial()`* / *src/fizzbuzz.rs* — *`fizzbuzz()`*

Like I'm 5

A loop is like telling someone "say hello 10 times." You tell them how many times, or you tell them "keep going until I say stop."

Like I'm 15

Rust has three loop kinds:

- **`for`** — iterate over a range or collection (most common)
- **`while`** — repeat while a condition is true
- **`loop`** — repeat forever until a `break`

```rust
// from src/loops.rs — factorial using for loop
fn factorial(n: u64) -> u64 {
    let mut result = 1;
    for i in 1..=n {   // 1..=n means 1, 2, 3, ... n (inclusive)
        result *= i;
    }
    result
}

// from src/fizzbuzz.rs — classic loop with conditionals
fn fizzbuzz(n: u32) -> String {
    if n % 15 == 0 { "FizzBuzz".to_string() }
    else if n % 3 == 0 { "Fizz".to_string() }
    else if n % 5 == 0 { "Buzz".to_string() }
    else { n.to_string() }
}
```

`1..5` is exclusive (1,2,3,4). `1..=5` is inclusive (1,2,3,4,5).

Professional

`for` iterates over anything that implements `IntoIterator`. `loop` is the only loop construct that can return a value via `break value`. Prefer `for` over `while` when iterating collections — harder to have off-

by-one errors and the borrow checker integrates cleanly. `for` consumes the iterator; use `.iter()` / `.iter_mut()` / `.into_iter()` explicitly to control ownership semantics.

---

11. Match Expressions

> *Source:* *src/match_expr.rs* — *see* *coin_value()*

Like I'm 5

Match is like a **sorting machine at a post office**. Each package comes in, and the machine checks: "Is this for house 1? House 2? Anything else?" Every package gets handled — you can't ignore any.

Like I'm 15

`match` checks a value against a list of patterns. Every possible case must be covered — the compiler will reject it if you miss one. The `_` wildcard catches anything not explicitly listed.

```
// from src/match_expr.rs
fn coin_value(coin: &str) -> u32 {
    match coin {
        "penny"   => 1,
        "nickel"  => 5,
        "dime"    => 10,
        "quarter" => 25,
        _         => 0,   // wildcard: anything else = 0
    }
}
```

`match` is also an expression — it produces a value. Both sides of each `=>` must be the same type.

Professional

`match` is Rust's most powerful control flow construct. It supports: literal patterns, range patterns (`1..=10`), struct and enum destructuring, guards (`if condition`), binding with `@` , and or-patterns (`A | B`). The compiler enforces exhaustiveness — adding a new enum variant surfaces all unhandled match sites. Use it over long `if` / `else if` chains whenever you're discriminating on a value.

---

12. Ownership

> *Source:* *src/ownership.rs* — *see* *join_strings()*

Like I'm 5

Every toy belongs to **exactly one kid**. When you give your toy to a friend, you don't have it anymore. Rust works the same way — every value has exactly one owner, and when that owner is done, the value gets **cleaned up automatically** — no trash collector, no manual cleanup.

Like I'm 15

Every value in Rust has one owner. When the owner goes out of scope (leaves the `{}`), the value is dropped — memory freed. Assigning a heap value (like `String`) to another variable **moves** it — the original

can't be used anymore.

```
// from src/ownership.rs
fn join_strings(s1: String, s2: String) -> String {
    s1 + " " + &s2   // s1 is moved into the + operation
                     // s2 is borrowed (&s2) so it's still accessible
}

let a = String::from("Hello");
let b = String::from("World");
let c = join_strings(a, b);
// a is gone now — it was moved into join_strings
// b is also gone — it was moved into join_strings
println!("{}", c); // "Hello World"
```

This is the key difference between heap types (`String`) and simple types (`i32`): numbers are so small they just get **copied** silently. Strings are moved because copying could be expensive.

Professional

Ownership rules: (1) each value has one owner, (2) only one owner at a time, (3) when the owner goes out of scope, the value is dropped (RAII). Types implementing `Copy` (all primitives, tuples/arrays of `Copy` types) are bitwise-copied on assignment — the original remains valid. `Clone` is an explicit deep copy. The move/copy distinction is a fundamental API design concern — decide early whether a type should be `Copy` .

---

13. Borrowing

> *Source:* *src/borrowing.rs* — *see* *count_chars()*

Like I'm 5

Borrowing is like **lending your toy to a friend**. They can play with it, but they give it back. You still own it — you just let them look at it for a while.

Like I'm 15

Instead of moving a value (and losing access), you can lend it with `&` . The function can read the value but can't take ownership. The `count_chars` function borrows a string to count its characters:

```
// from src/borrowing.rs
fn count_chars(s: &str) -> usize {
    s.chars().count()  // we just read s, we don't own it
}

let name = String::from("Alice");
let n = count_chars(&name); // lend name to the function
println!("{} has {} chars", name, n); // name still works!
```

`&name` means "a reference to name" — the `&` is how you lend something.

Professional

Immutable borrows (`&T`) allow multiple simultaneous readers. The borrow checker guarantees: any number of `&T` OR exactly one `&mut T` , never both at the same time. This eliminates data races at compile time. Prefer `&str` over `&String` in function parameters — `&str` is more general (accepts literals and slice expressions without allocation). NLL (Non-Lexical Lifetimes) tracks borrows at the point of last use, not end of scope.

---

14. Mutable Borrowing

> *Source:* *src/mut_borrowing.rs* — *see* *double_all()*

Like I'm 5

Mutable borrowing is like letting a friend borrow your **whiteboard AND letting them write on it**. But only one person can write at a time, and nobody else can even look while they're writing.

Like I'm 15

`&mut T` lets a function modify a value without taking ownership. Only one mutable reference can exist at a time — and no immutable references can exist either while you have one.

```
// from src/mut_borrowing.rs
fn double_all(numbers: &mut Vec<i32>) {
    for n in numbers.iter_mut() {
        *n *= 2;  // * means "follow the reference and change the actual value"
    }
}

let mut nums = vec![1, 2, 3];
double_all(&mut nums); // lend nums for modification
println!("{:?}", nums); // [2, 4, 6] — the original was changed!
```

The `*` in `*n *= 2` is the **dereference** operator — it follows the reference arrow and operates on the real value, not the reference itself.

Professional

The exclusive mutable reference rule prevents iterator invalidation, dangling pointers, and data races — all at compile time. The dereference operator `*` is needed to assign through a `&mut` reference; Rust auto-derefs for method calls (`.` syntax). Interior mutability patterns (`RefCell` , `Mutex` , `AtomicXxx`) permit mutation through shared references at the cost of runtime checks or synchronization overhead.

---

15. Structs

> *Source:* *src/structs.rs* — *see* *Rectangle* , *square()* , *area()* , *is_square()*

Like I'm 5

A struct is like a **form you fill out** about something: name, age, favorite color. All the info about one thing, bundled together and given a name.

Like I'm 15

Structs group related data under one type. You add behavior with `impl` blocks. The source file builds a `Rectangle` struct with methods:

```
// from src/structs.rs
struct Rectangle {
    width: f64,
    height: f64,
}

impl Rectangle {
    // Associated function (no self) — a constructor
    fn square(size: f64) -> Self {
        Rectangle { width: size, height: size }
    }

    // Method: reads the struct (&self = immutable borrow)
    fn area(&self) -> f64 {
        self.width * self.height
    }

    // Method: reads the struct to check something
    fn is_square(&self) -> bool {
        self.width == self.height
    }
}

let r = Rectangle::square(5.0); // :: calls the associated function
println!("{}", r.area());       // . calls the method
```

- `&self` = "I only need to read the struct"
- `&mut self` = "I need to change the struct"
- `self` = "I consume the struct (it's gone after)"
- No `self` = it's a constructor / static helper, called with `::`

Professional

Structs are nominal types — two structurally identical structs are distinct types. `impl` blocks can be split across a file; only one `impl Trait for Struct` per trait. `Self` inside `impl` aliases the implementing type — use it in constructors to avoid breaking code when renaming. The `..other` struct update syntax copies remaining fields. Newtype pattern (`struct Meters(f64)`) adds type safety with zero runtime cost. Always `#[derive(Debug)]` during development.

---

16. Enums

> *Source:* *`src/enums.rs`* *— see* *`get_grade()`*

Like I'm 5

An enum is like a **traffic light**: it can only be one color at a time — red, yellow, or green. Nothing else is allowed. You always know exactly what states are possible.

Like I'm 15

Enums define a type that can be one of several named variants. The `get_grade` function uses range patterns inside a match to convert a score to a grade:

```
// from src/enums.rs
fn get_grade(score: u32) -> &'static str {
    match score {
        90..=100 => "A",
        80..=89  => "B",
        70..=79  => "C",
        60..=69  => "D",
        _        => "F",
    }
}
```

Enums can also carry data inside each variant:

```
enum Message {
    Quit,                       // no data
    Move { x: i32, y: i32 },   // named fields
    Write(String),              // holds a String
}
```

Professional

Rust enums are algebraic data types (sum types). Variants can be unit, tuple, or struct style. The discriminant is stored efficiently (often a single byte). Enums model state machines well — the compiler enforces exhaustiveness. Adding a new variant is a breaking change that surfaces all unhandled match sites. Prefer enums over boolean flags or string constants wherever the set of cases is finite and known.

---

17. Option<T>

> *Source:* `src/option.rs` *— see* `safe_divide()`

Like I'm 5

`Option` is like a **gift box**. It's either a `Some` box with something inside, or a `None` box that's completely empty. You have to **check before you can use** what's inside — you can't just assume there's something there.

Like I'm 15

`Option<T>` is Rust's replacement for `null`. It has two variants: `Some(value)` meaning "there is a value" and `None` meaning "there isn't." The `safe_divide` function uses it to handle division by zero without crashing:

```rust
// from src/option.rs
fn safe_divide(a: f64, b: f64) -> Option<f64> {
    if b == 0.0 {
        None          // no result — can't divide by zero
    } else {
        Some(a / b)   // wrap the result in Some
    }
}

match safe_divide(10.0, 2.0) {
    Some(result) => println!("Result: {}", result),
    None         => println!("Cannot divide by zero"),
}
```

Professional

`Option<T>` eliminates null pointer exceptions at the type level. Key combinators: `.map()`, `.and_then()` (flatMap), `.unwrap_or()`, `.unwrap_or_else()`, `.ok_or()` (converts to Result). Use `?` inside functions returning `Option` — it short-circuits on `None`. Avoid `.unwrap()` in production unless the `None` case is provably impossible. `Option<Box<T>>` has the same size as `*mut T` due to niche optimization — `None` maps to null pointer, zero overhead.

---

18. Result<T, E>

> *Source:* *src/result.rs* — *see* *parse_number()*

Like I'm 5

`Result` is like a **vending machine**. You put in money and either get a snack (`Ok`) or get your money back with an error message on screen (`Err`). You always have to check which one you got.

Like I'm 15

`Result<T, E>` represents an operation that either succeeds with a value (`Ok(T)`) or fails with an error (`Err(E)`). The `parse_number` function converts a string to a number and handles failure:

```rust
// from src/result.rs
fn parse_number(s: &str) -> Result<i32, String> {
    s.trim()
     .parse::<i32>()
     .map_err(|e| e.to_string())   // convert the error to a String
}

match parse_number("42") {
    Ok(n)  => println!("Got: {}", n),
    Err(e) => println!("Failed: {}", e),
}
```

`Result` vs `Option`:

- `Option` = value might not exist (no reason why)
- `Result` = operation might fail (and you get a reason: the error)

Professional

`Result<T, E>` is the foundation of Rust's error handling. `E` can be `String` for prototypes, a custom enum for production, `Box<dyn std::error::Error>` for heterogeneous errors, or library types via `thiserror` / `anyhow`. Combinators: `.map()`, `.map_err()`, `.and_then()`, `.or_else()`. The `?` operator desugars to early-return with `From::from(e)` conversion — design error types to implement `From` for seamless composition.

---

19. The `?` Operator

> *Sources:* *src/question_mark.rs* — *`add_parsed()`* / *src/csv_parse.rs* — *`parse_csv_sum()`*

Like I'm 5

Imagine you're building a tower of blocks. If any block falls, the whole tower stops immediately. The `?` is like a helper that watches each block — if it fails, it immediately says "sorry, the tower fell" and **stops right there**, rather than trying to keep going on a broken foundation.

Like I'm 15

`?` is shorthand for "if this is an error, return that error immediately." It makes error-propagating code clean without `match` everywhere.

```rust
// from src/question_mark.rs
fn add_parsed(a: &str, b: &str) -> Result<i32, String> {
    let x = a.trim().parse::<i32>().map_err(|e| e.to_string())?;
    //                                                      ^
    // If parse() fails, ? returns the Err immediately.
    // If it succeeds, x gets the number and we continue.

    let y = b.trim().parse::<i32>().map_err(|e| e.to_string())?;
    Ok(x + y)
}
```

```rust
// from src/csv_parse.rs — chaining multiple ? operations
fn parse_csv_sum(input: &str) -> Result<i32, String> {
    if input.is_empty() { return Err("empty input".to_string()); }
    let mut total = 0;
    for part in input.split(',') {
        let n = part.trim().parse::<i32>()
            .map_err(|_| format!("bad value: '{}'", part.trim()))?;
        total += n;
    }
    Ok(total)
}
```

Professional

`?` desugars to `match result { Ok(v) => v, Err(e) => return Err(From::from(e)) }`. The `From::from(e)` conversion allows composing different error types as long as the function's `Err` implements `From<OriginalError>`. Works on both `Result` and `Option`. Can only be used in functions returning a compatible type. `main` can return `Result<(), E>` to allow `?` at the top level.

---

20. Vectors

> *Source:* *src/vectors.rs* — *see* *sum_vec()*

Like I'm 5

A vector is like a **magical backpack that can grow** when you add more stuff. An array has a fixed size you decide upfront; a vector can stretch as big as you need.

Like I'm 15

`Vec<T>` is a growable list stored on the **heap**. It has a pointer (on the stack) pointing to the actual data (on the heap), plus a length and a capacity.

```
// from src/vectors.rs
fn sum_vec(v: &[i32]) -> i32 {
    let mut total = 0;
    for &n in v {
        total += n;
    }
    total
}

let mut nums = vec![1, 2, 3];   // create with values
nums.push(4);                    // add to the end
nums.pop();                      // remove from the end → returns Option<i32>

println!("{}", nums[0]);         // index access — panics if out of bounds
println!("{:?}", nums.get(10));  // safe access — returns None instead of panic
```

Professional

`Vec<T>` owns a heap allocation: pointer, length, and capacity. Pushing beyond capacity triggers reallocation (amortized O(1)). Use `Vec::with_capacity(n)` when the final size is known. `.get(i)` returns `Option<&T>` (safe); `v[i]` panics on out-of-bounds (use only when you've verified bounds). Pass `&[T]` to functions, not `&Vec<T>` — it's more general (accepts arrays too via deref coercion). `drain`, `retain`, and `dedup` are workhorses for in-place filtering.

---

21. Strings

> *Source:* *src/strings.rs* — *see* *count_vowels()*

Like I'm 5

`String` is like a **stretchy ribbon** you can write on and change. `&str` is like a **printed label** — you can read it but not change it, and it's attached to whatever it came from.

Like I'm 15

Rust has two string types:

- **`&str`** — a view into some text, can't grow, usually a literal
- **`String`** — owns its text, lives on the heap, can grow

```
let greeting = "hello";                 // &str — hardcoded, lives forever
let mut s = String::from("hello");      // String — owned, heap-allocated
s.push_str(", world");                  // can be changed

// from src/strings.rs
fn count_vowels(s: &str) -> usize {
    s.chars()                           // iterate over Unicode characters
     .filter(|c| "aeiouAEIOU".contains(*c))  // keep only vowels
     .count()
}
```

Important: Rust strings are always valid UTF-8. You can't index by position (`s[0]`) because a character might be 1–4 bytes. Use `.chars()` to iterate characters safely.

Professional

`.chars()` yields Unicode scalar values. `.bytes()` yields raw `u8` bytes. `.split()` , `.split_whitespace()` , `.lines()` return `&str` slices into the original (zero allocation). String `+` moves the left `String` and borrows the right `&str` . For building strings in loops, `String::with_capacity` + `push_str` beats repeated `+` . For complex formatting, use `format!` .

---

22. HashMaps & HashSets

> *Source:* *`src/hashmaps.rs`* — *see* *`unique_word_count()`*

Like I'm 5

A **HashMap** is like a locker room. Each locker has a **name tag** (key) and holds **stuff** (value). You find things by name, not by counting doors. A **HashSet** is like a list where every name can only appear **once** — no duplicates allowed.

Like I'm 15

`HashMap<K, V>` stores key-value pairs with fast lookup. `HashSet<T>` is a collection with unique values only.

```
// from src/hashmaps.rs
use std::collections::HashSet;

fn unique_word_count(text: &str) -> usize {
    let words: HashSet<&str> = text.split_whitespace().collect();
```

```
    //                                                  ^
    // .collect() gathers the iterator into a HashSet
    // duplicate words are automatically removed
    words.len()
}
```

For HashMaps:

```
use std::collections::HashMap;

let mut scores: HashMap<&str, i32> = HashMap::new();
scores.insert("Alice", 100);
scores.entry("Bob").or_insert(0);        // insert 0 if Bob isn't there yet
*scores.entry("Alice").or_insert(0) += 10; // update Alice's score in place
```

The `.entry().or_insert()` pattern is how you say "add this key if it doesn't exist, then give me its value."

Professional

`HashMap` uses SipHash by default (DoS-resistant, not the fastest). For performance-critical code, `FxHashMap` or `AHashMap` from crates are faster. `.entry()` avoids double lookup — the idiomatic insert-or-update. `.or_insert()` returns `&mut V` for in-place modification. `HashSet` is `HashMap<T, ()>` with set operations: `union`, `intersection`, `difference`. Keys must implement `Hash + Eq`. Iteration order is non-deterministic — use `BTreeMap` / `BTreeSet` for sorted order.

---

23. Traits

> *Source:* *src/traits.rs* — *see* `Describable`, `Item`, `describe()`

Like I'm 5

A trait is like a **job description**. If you want to be a "singer," you must be able to sing. If you want to be a "swimmer," you must be able to swim. Traits say what abilities a type must have before you can use it in certain ways.

Like I'm 15

A trait defines a set of methods that a type must implement. Any type that implements the trait gets those abilities. The source file defines a `Describable` trait:

```
// from src/traits.rs
trait Describable {
    fn describe(&self) -> String;
}

struct Item {
    name: String,
    value: f64,
}
```

```rust
impl Describable for Item {
    fn describe(&self) -> String {
        format!("{} costs ${:.2}", self.name, self.value)
    }
}

let thing = Item { name: "Book".to_string(), value: 12.99 };
println!("{}", thing.describe()); // "Book costs $12.99"
```

Traits are similar to "interfaces" in other languages — but more powerful. Any type, even ones you didn't write, can implement your trait.

Professional

Traits are Rust's ad-hoc polymorphism mechanism: static dispatch (generic `T: Trait` — monomorphized, zero cost) vs dynamic dispatch (`dyn Trait` — vtable pointer, runtime cost). Trait coherence: you can only implement a trait for a type if you own the trait OR the type (orphan rule). Default method implementations allow optional override. Marker traits (`Copy` , `Send` , `Sync`) carry no methods but convey semantic guarantees the compiler enforces.

---

24. Derive Macros

> *Source: src/derive.rs — see `Point` , `are_equal()` , `distance_sq()`*

Like I'm 5

Derive is like a **magic stamp**. Instead of teaching your struct every rule by hand, you just stamp `#[derive(Debug)]` on it and it automatically knows how to show itself on screen, be copied, be compared, and more.

Like I'm 15

`#[derive]` automatically generates common trait implementations. Without it, you'd have to write them yourself — tedious and error-prone.

```rust
// from src/derive.rs
#[derive(Debug, Clone, PartialEq)]
struct Point {
    x: f64,
    y: f64,
}

fn are_equal(a: &Point, b: &Point) -> bool {
    a == b  // works because of PartialEq derive
}

fn distance_sq(a: &Point, b: &Point) -> f64 {
    let dx = a.x - b.x;
    let dy = a.y - b.y;
    dx * dx + dy * dy
}
```

```rust
let p1 = Point { x: 1.0, y: 2.0 };
let p2 = p1.clone();          // works because of Clone derive
println!("{:?}", p1);         // works because of Debug derive
assert!(are_equal(&p1, &p2)); // works because of PartialEq derive
```

Common derives:

- `Debug` → print with `{:?}` (for debugging)
- `Clone` → `.clone()` makes a deep copy
- `PartialEq` → `==` and `!=` comparisons
- `Hash` → can be used as a HashMap key

Professional

Derive macros expand at compile time via procedural macros. Derived `PartialEq` compares all fields field-by-field — override with a manual `impl` when needed. `Hash` and `Eq` must be consistent: `a == b` implies `hash(a) == hash(b)`. `Copy` is only derivable if all fields are `Copy`. Third-party derives (`serde::Serialize`, `thiserror::Error`) follow the same mechanism.

---

25. Generics

> *Source:* *src/generics.rs* — *see* *`largest()`*

Like I'm 5

Imagine a **magical container** that can hold anything — toys, apples, books. You tell it what you'll put inside, and it shapes itself to fit. Generics let you write code that works for many types without copying the same code over and over.

Like I'm 15

Generics let you write a function or type that works for any type `T`. Trait bounds (`T: SomeTrait`) restrict what `T` can be so you can call methods on it. The `largest` function works for any type that can be compared:

```rust
// from src/generics.rs
fn largest<T: PartialOrd>(list: &[T]) -> &T {
    //      ^              ^ T must be comparable with >
    let mut biggest = &list[0];
    for item in list {
        if item > biggest {
            biggest = item;
        }
    }
    biggest
}

// Works with numbers:
let nums = vec![34, 50, 25, 100, 65];
println!("{}", largest(&nums)); // 100
```

```rust
// Works with chars:
let chars = vec!['y', 'm', 'a', 'q'];
println!("{}", largest(&chars)); // y
```

Professional

Rust generics are monomorphized — the compiler generates a concrete copy for each used type, eliminating virtual dispatch. Trait bounds can be inline (`T: Trait + OtherTrait`) or in `where` clauses. Lifetime parameters are also generic: `fn foo<'a>(x: &'a str) -> &'a str` . High monomorphization cost (large binaries, slow compilation) is the tradeoff — use `dyn Trait` when heterogeneous dispatch is needed. `PhantomData<T>` handles unused generic parameters without runtime cost.

---

26. Lifetimes

> *Sources:* *src/lifetimes.rs* — *`longest()`* | *src/lifetime_annotations.rs* — *`trim_prefix()`* | *src/elision.rs*

Like I'm 5

Imagine a **library book**. You can borrow it, but you have to return it before the library closes. Lifetimes make sure you never try to use a book that's already been put back on the shelf — they prevent you from holding a reference to something that no longer exists.

Like I'm 15

Lifetimes tell the compiler how long a reference is valid. They're written with `'a` (tick + a name). The `longest` function needs lifetimes because it returns a reference to one of its inputs, and the compiler needs to know: "how long is the returned reference valid?"

```rust
// from src/lifetimes.rs
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    //     ^^^              ^^^            ^^^
    // 'a means: "the return value lives at least as long as both inputs"
    if x.len() > y.len() { x } else { y }
}
```

```rust
// from src/elision.rs — lifetimes can often be left out (elided)
fn trim_prefix(s: &str, prefix: &str) -> &str {
    // compiler figures out the lifetime automatically here
    if s.starts_with(prefix) { &s[prefix.len()..] } else { s }
}
```

Most of the time you don't need to write lifetimes because the compiler figures them out automatically. You only need to write them explicitly when the compiler can't.

Professional

Lifetimes are compile-time annotations with zero runtime representation. The borrow checker uses them to ensure no reference outlives its referent. Elision rules: (1) each input reference gets its own lifetime, (2) if exactly one input lifetime, it's assigned to all outputs, (3) if one input is `&self` / `&mut self` , its lifetime

goes to all outputs. `'static` means the reference is valid for the entire program duration (string literals, `Box::leak`). Struct lifetime annotations (`struct Foo<'a> { s: &'a str }`) express that the struct cannot outlive the borrowed data.

---

27. Closures

> *Source:* *src/closures.rs* — *see* *double_all()*

Like I'm 5

A closure is like a **little recipe card** you can hand to someone else to use later. It can remember ingredients from your kitchen (its surroundings) even when it's being used somewhere far away.

Like I'm 15

Closures are anonymous functions defined inline. They can **capture** variables from the surrounding code. Syntax: `|params| expression`.

```
// from src/closures.rs
fn double_all(numbers: &[i32]) -> Vec<i32> {
    numbers.iter()
           .map(|&n| n * 2)   // |&n| n * 2 is a closure: takes n, returns n*2
           .collect()
}

// Closures can capture from outside:
let multiplier = 3;
let triple = |x| x * multiplier;  // captures multiplier from the outer scope
println!("{}", triple(5));         // 15
```

Professional

Closures implement one of: `Fn` (multiple calls, captures by shared reference), `FnMut` (multiple calls, captures by mutable reference), `FnOnce` (single call, captures by value). The compiler infers which — prefer `Fn` in bounds. `move` closures take ownership of captured variables (required for `thread::spawn`). Closures are sized types — store them in generics for zero cost or `Box<dyn Fn()>` for heterogeneous collections.

---

28. Iterators

> *Sources:* *src/iterators.rs* — *sum_of_squares()* | *src/filter.rs* — *evens_only()*

Like I'm 5

An iterator is like a **conveyor belt in a factory**. Items come one at a time. You can put a **filter** in the middle to skip some, a **transformer** to change each item, and a **bucket** at the end to collect everything. The belt doesn't move until someone turns it on — that's the "lazy" part.

Like I'm 15

Iterators process collections lazily — nothing actually runs until you call a "consuming" method at the end. You chain adapters and finish with a consumer.

```
// from src/iterators.rs
fn sum_of_squares(numbers: &[i32]) -> i32 {
    numbers.iter()           // create an iterator over the slice
           .map(|&n| n * n)  // transform: square each number
           .sum()            // consume: add them all up → this triggers everything
}

// from src/filter.rs
fn evens_only(numbers: &[i32]) -> Vec<i32> {
    numbers.iter()
           .filter(|&&n| n % 2 == 0)  // keep only even numbers
           .copied()                  // convert &i32 references to i32 values
           .collect()                 // gather into a Vec
}
```

Adapters (lazy, do nothing yet): `.map()`, `.filter()`, `.take()`, `.skip()`, `.enumerate()`
Consumers (triggers everything): `.sum()`, `.collect()`, `.count()`, `.max()`, `.for_each()`

Professional

The `Iterator` trait requires one method: `fn next(&mut self) -> Option<Self::Item>`. All 70+ other methods are provided for free. Iterators are lazy — the chain is a description of computation; a terminal method triggers execution. The optimizer fuses operations, eliminating intermediate allocations. Prefer iterator chains over index-based loops — safer (no out-of-bounds) and often faster due to LLVM fusion.

---

29. Custom Iterators

> *Source:* *src/custom_iter.rs* — *see* *`Countdown`*, *`new()`*, *`next()`*

Like I'm 5

You can **build your own conveyor belt**. You decide what items go on it and when. You just need to teach it one thing: "what comes next?"

Like I'm 15

You can implement `Iterator` for your own types by defining a `next` method. Once you do, all the standard adapters (`.map()`, `.filter()`, `.collect()`) work automatically.

```
// from src/custom_iter.rs
struct Countdown {
    count: u32,
}

impl Countdown {
    fn new(start: u32) -> Self {
        Countdown { count: start }
    }
```

```
}

impl Iterator for Countdown {
    type Item = u32;    // what type of items does this iterator produce?

    fn next(&mut self) -> Option<u32> {
        if self.count > 0 {
            self.count -= 1;
            Some(self.count + 1)  // yield the current count
        } else {
            None  // signal: we're done
        }
    }
}

// Once you implement Iterator, all adapters work for free:
let sum: u32 = Countdown::new(5).sum(); // 5+4+3+2+1 = 15
let evens: Vec<u32> = Countdown::new(10).filter(|n| n % 2 == 0).collect();
```

Professional

Implementing `Iterator` grants all 70+ methods via blanket implementations. `type Item` is the associated type for element type. Return `None` to signal exhaustion — iterators are fused by convention (after `None`, always `None`). Implement `ExactSizeIterator` to provide `.len()`, enabling `collect` to pre-allocate. `IntoIterator` is what `for` loops call implicitly — implement it to make your type directly iterable in a `for` loop without calling `.iter()`.

---

30. Associated Types

> *Source:* *src/assoc_types.rs* — see *`Summarize`*, *`Numbers`*, *`Sentence`*

Like I'm 5

An associated type is like a **fill-in-the-blank on a job form**. The trait says "you must have an `Output` type — write what yours is." Each type that does the job fills in its own answer.

Like I'm 15

Associated types let a trait say "there is a type related to this implementation, but I'll let the implementor decide what it is." The `Summarize` trait in the source uses this:

```
// from src/assoc_types.rs
trait Summarize {
    type Output;                    // blank to fill in
    fn summarize(&self) -> Self::Output;
}

struct Numbers(Vec<i32>);
struct Sentence(String);

impl Summarize for Numbers {
```

```
    type Output = i32;             // Numbers summarizes to a number (the sum)
    fn summarize(&self) -> i32 {
        self.0.iter().sum()
    }
}

impl Summarize for Sentence {
    type Output = usize;           // Sentence summarizes to a count (word count)
    fn summarize(&self) -> usize {
        self.0.split_whitespace().count()
    }
}
```

Professional

Associated types vs generics: generics allow multiple implementations (`impl Add<i32> for Foo` and `impl Add<f64> for Foo`); associated types allow exactly one (`impl Iterator for Foo` has one `Item`). Use associated types when the relationship between implementor and related type is one-to-one. Bounds with associated types: `T: Iterator<Item = i32>` is cleaner than an extra generic. Resolved at compile time — no runtime cost.

Capstone: Three Projects That Use Everything Together

> *Sources:* *src/capstone.rs* / *src/diagonal.rs* / *src/calculator.rs*

`most_frequent()` — src/capstone.rs

Uses: `HashMap` , `.entry().or_insert()` , iterators, closures, `.find()` , `.to_lowercase()`

Finds the most common word in a string. This is the most "real world" function in the whole repo — it's how you'd actually count word frequency in a program.

`diagonal_sum()` — src/diagonal.rs

Uses: nested `Vec` , `.split()` , `.map()` , `.collect()` , multi-level iteration

Parses a text representation of a 2D matrix and sums the diagonal. Shows how iterator chains compose for complex parsing.

`evaluate()` — src/calculator.rs

Uses: `Result` , `?` , `match` on operators, division by zero handling, string parsing

A full expression evaluator: parses `"10 + 5"` → `Ok(15)` , `"10 / 0"` → `Err(...)` . Every error handling concept in one function.

Quick Reference: Mental Models

Concept	Core Idea	Source File
Stack	Fast, automatic, fixed-size values	(everywhere)

Heap	Flexible, manual, variable-size values	`ownership.rs`, `vectors.rs`
`&`	A reference — address of a value, not the value itself	`borrowing.rs`
Ownership	One owner. Owner drops = value freed.	`ownership.rs`
Borrowing	Lend without giving up ownership. Many readers OR one writer.	`borrowing.rs`, `mut_borrowing.rs`
Lifetime	How long is this reference valid?	`lifetimes.rs`, `elision.rs`
Option	Value might not exist. Safer than null.	`option.rs`
Result	Operation might fail. Forces you to handle it.	`result.rs`, `csv_parse.rs`
`?`	Return the error immediately if it fails.	`question_mark.rs`
Trait	What can this type do?	`traits.rs`
Generic	Write once, works for many types.	`generics.rs`
Iterator	Lazy conveyor belt. Nothing runs until consumed.	`iterators.rs`, `filter.rs`
Closure	Portable mini-function that remembers its surroundings.	`closures.rs`
Enum	A type that is exactly one of a fixed set of variants.	`enums.rs`, `option.rs`
Derive	Auto-generate common behaviors with a stamp.	`derive.rs`
Associated Type	A type that belongs to a trait implementation.	`assoc_types.rs`